



南京大學

NANJING UNIVERSITY

链路层和局域网

孟晴开

智能软件与工程学院

苏州校区南雍楼西区228

qkmeng@nju.edu.cn , <https://mengqingkai.github.io/>



Outline

- Introduction
- Error detection, correction
- Multiple access protocols
- LANs
- Link virtualization: MPLS
- Data center networking
- A day in the life of a web request



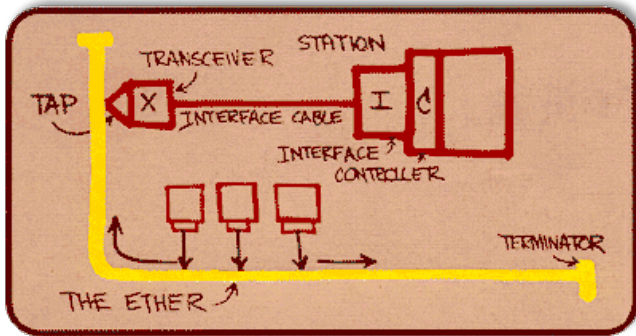


Ethernet

“dominant” wired LAN technology:

- first widely used LAN technology
- simpler, cheap
- kept up with speed race: 10 Mbps - 400 Gbps
- single chip, multiple speeds (e.g., Broadcom BCM5761)

Metcalfe's Ethernet sketch



Bob Metcalfe: Ethernet co-inventor,
2022 ACM Turing Award recipient

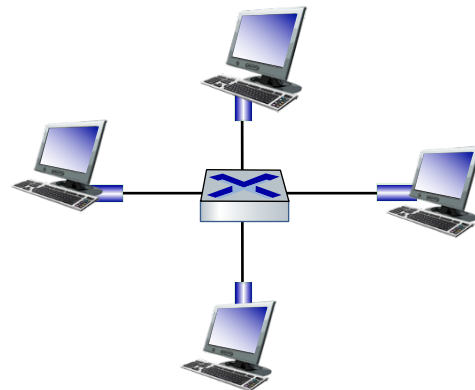




Ethernet: physical topology

- **bus:** popular through mid 90s
 - all nodes in same collision domain (can collide with each other)
- **switched:** prevails today
 - active link-layer 2 **switch** in center
 - each "spoke" runs a (separate) Ethernet protocol (nodes do not collide with each other)

bus: coaxial cable



switched





Ethernet frame structure

sending interface encapsulates IP datagram (or other network layer protocol packet) in **Ethernet frame**



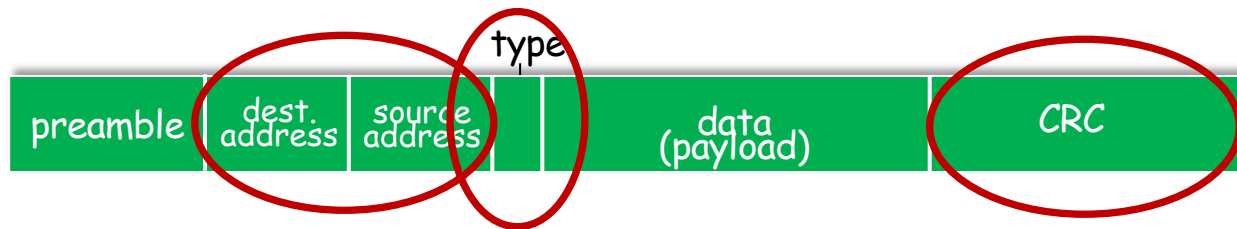
preamble:

- used to synchronize receiver, sender clock rates
- 7 bytes of 10101010 followed by one byte of 10101011





Ethernet frame structure (more)



- **addresses:** 6 byte source, destination MAC addresses
 - if adapter receives frame with matching destination address, or with broadcast address (e.g., ARP packet), it passes data in frame to network layer protocol
 - otherwise, adapter discards frame
- **type:** indicates higher layer protocol
 - mostly IP but others possible, e.g., Novell IPX, AppleTalk
 - used to demultiplex up at receiver
- **CRC:** cyclic redundancy check at receiver
 - error detected: frame is dropped





Ethernet: unreliable, connectionless

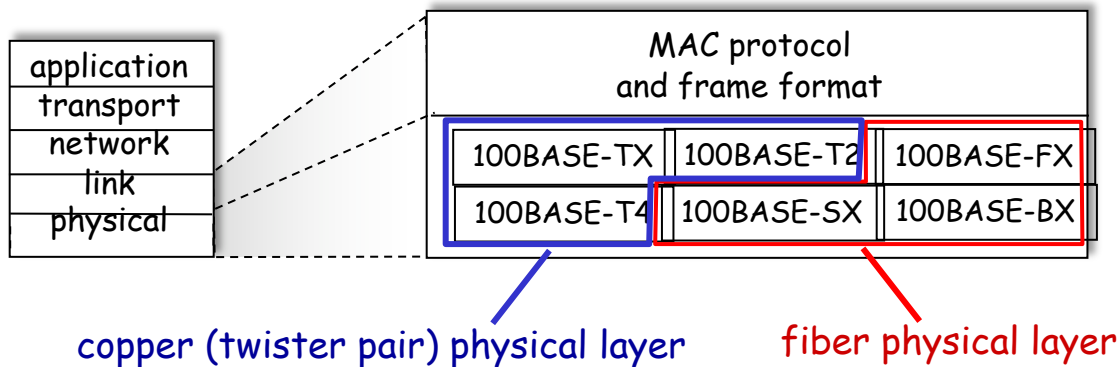
- **connectionless**: no handshaking between sending and receiving NICs
- **unreliable**: receiving NIC doesn't send ACKs or NAKs to sending NIC
 - data in dropped frames recovered only if initial sender uses higher layer rdt (e.g., TCP), otherwise dropped data lost
- Ethernet's MAC protocol: unslotted **CSMA/CD with binary backoff**





802.3 Ethernet standards: link & physical layers

- many different Ethernet standards
 - common MAC protocol and frame format
 - different speeds: 2 Mbps, ... 100 Mbps, 1Gbps, 10 Gbps, 40 Gbps, 80 Gbps
 - ✓ different physical layer media: fiber, cable





Ethernet switch

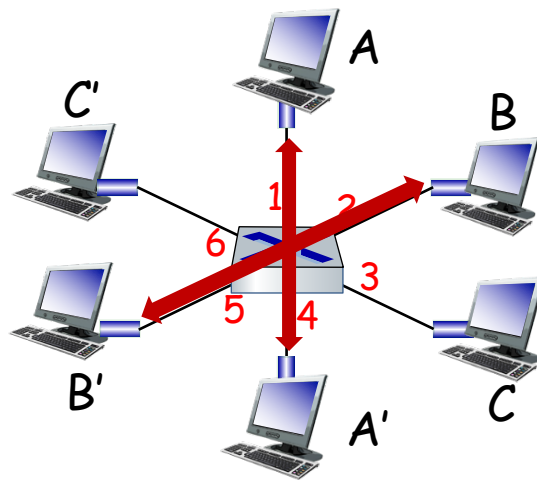
- Switch is a **link-layer** device: takes an **active** role
 - store, forward Ethernet (or other type of) frames
 - examine incoming frame's MAC address, **selectively** forward frame to one-or-more outgoing links when frame is to be forwarded on segment, uses CSMA/CD to access segment
- **transparent**: hosts unaware of presence of switches
- **plug-and-play, self-learning**
 - switches do not need to be configured





Switch: multiple simultaneous transmissions

- hosts have dedicated, direct connection to switch
- switches buffer packets
- Ethernet protocol used on each incoming link, so:
 - no collisions; full duplex
 - each link is its own collision domain
- **switching:** A-to-A' and B-to-B' can transmit simultaneously, without collisions



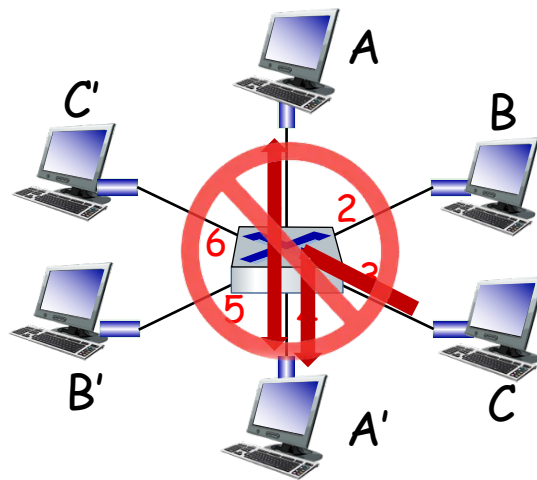
switch with six
interfaces
(1,2,3,4,5,6)





Switch: multiple simultaneous transmissions

- hosts have dedicated, direct connection to switch
- switches buffer packets
- Ethernet protocol used on each incoming link, so:
 - no collisions; full duplex
 - each link is its own collision domain
- **switching:** A-to-A' and B-to-B' can transmit simultaneously, without collisions
 - but A-to-A' and C to A' can not happen simultaneously



switch with six
interfaces
(1,2,3,4,5,6)





Switch forwarding table

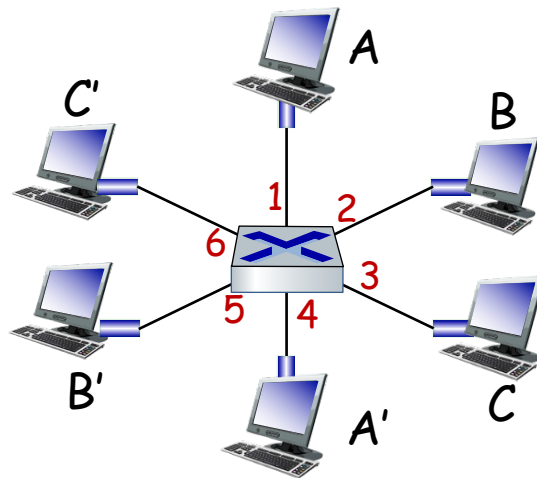
Q: how does switch know A' reachable via interface 4, B' reachable via interface 5?

A: each switch has a **switch table**, each entry:

- (MAC address of host, interface to reach host, time stamp)
- looks like a routing table!

Q: how are entries created, maintained in switch table?

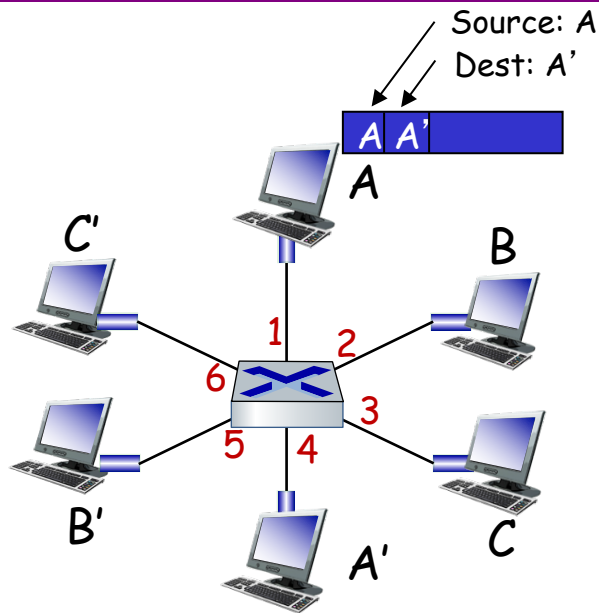
- something like a routing protocol?





Switch: self-learning

- switch **learns** which hosts can be reached through which interfaces
 - when frame received, switch “learns” location of sender: incoming LAN segment
 - records sender/location pair in switch table



MAC addr	interface	TTL
A	1	60

Switch table
(initially empty)





Switch: frame filtering/forwarding

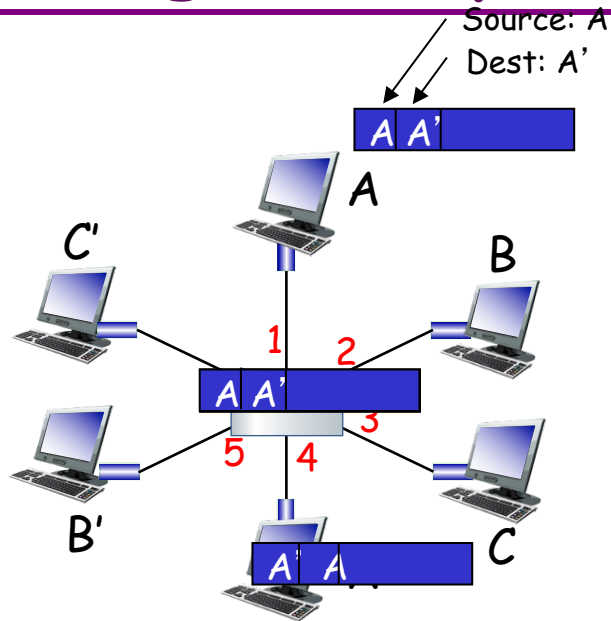
when frame received at switch:

1. record incoming link, MAC address of sending host
2. index switch table using MAC destination address
3. **if** entry found for destination
 then {
 if destination on segment from which frame arrived
 then drop frame
 else forward frame on interface indicated by entry
 }
 else flood /* forward on all interfaces except arriving interface */



Self-learning, forwarding: example

- frame destination, A' ,
location unknown: **flood**
- destination A location
known: **selectively send**
on just one link



MAC addr	interface	TTL
A	1	60
A'	4	60

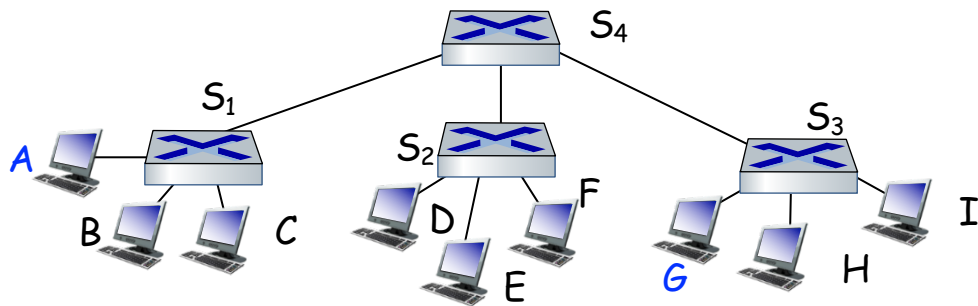
switch table
(initially empty)





Interconnecting switches

self-learning switches can be connected together:



Q: sending from A to G - how does S₁ know to forward frame destined to G via S₄ and S₃?

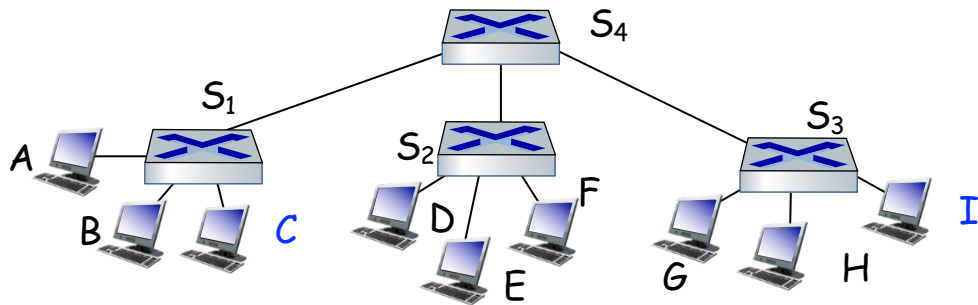
➤ A: self learning! (works exactly the same as in single-switch case!)





Self-learning multi-switch example

Suppose *C* sends frame to *I*, *I* responds to *C*

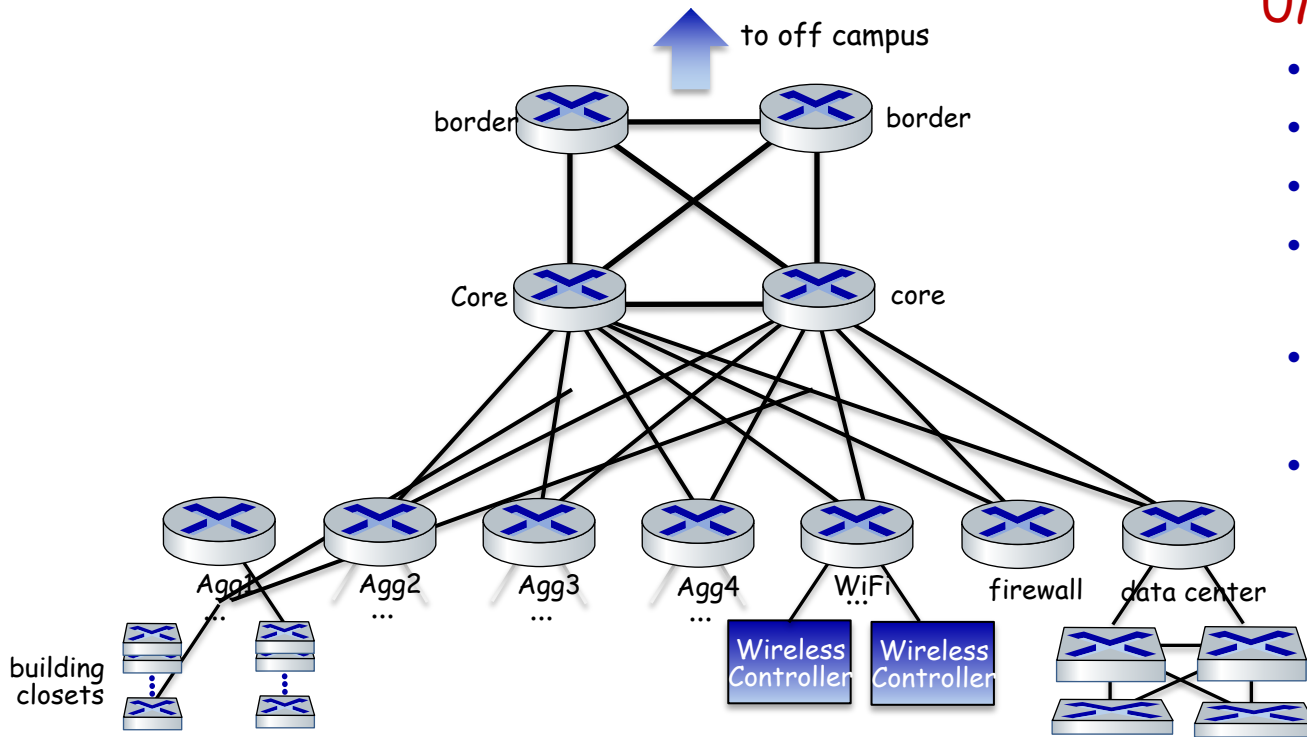


Q: show switch tables and packet forwarding in S_1 , S_2 , S_3 , S_4





UMass Campus Network - Detail



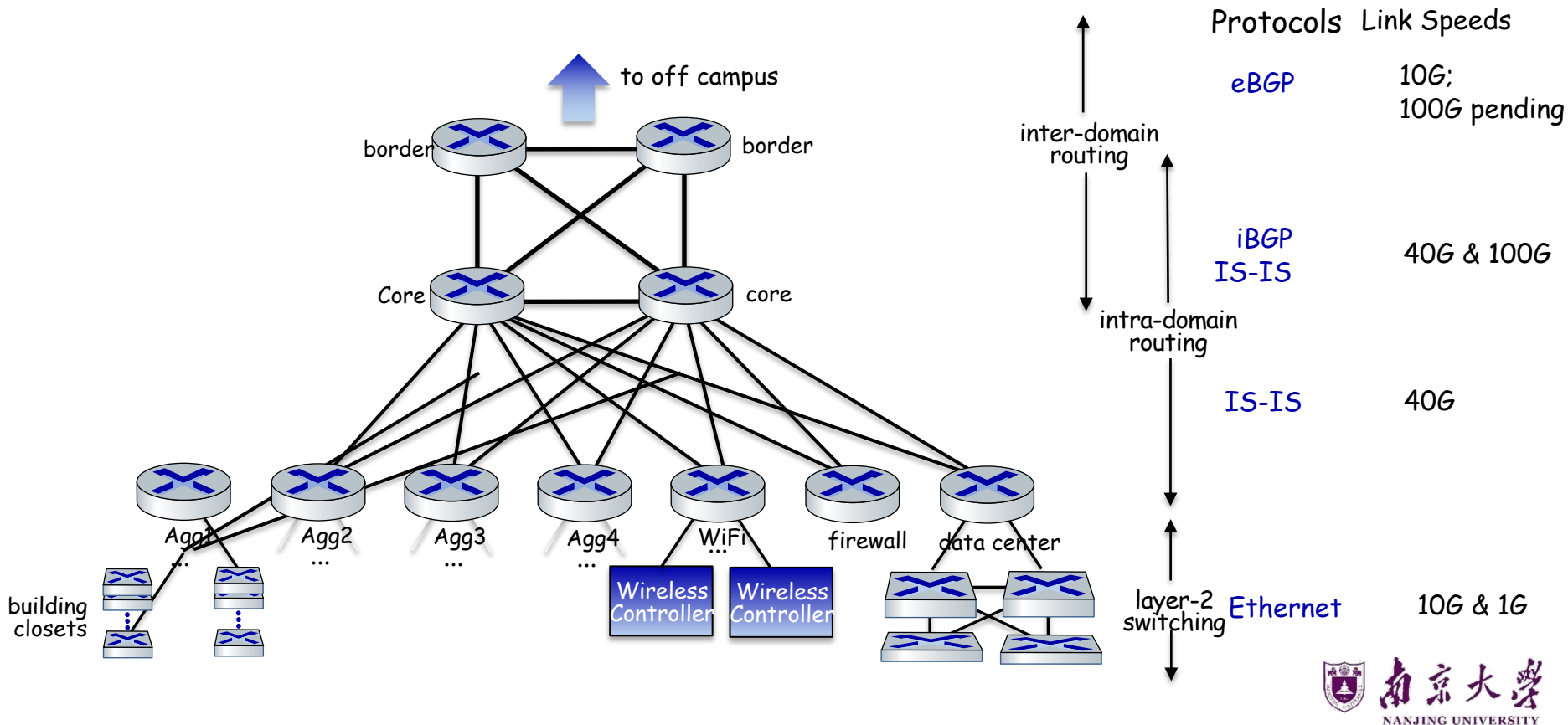
UMass network:

- 4 firewalls
- 10 routers
- 2000+ network switches
- 6000 wireless access points
- 30000 active wired network jacks
- 55000 active end-user wireless devices

... all built,
operated,
maintained by
~15 people



UMass Campus Network - Detail





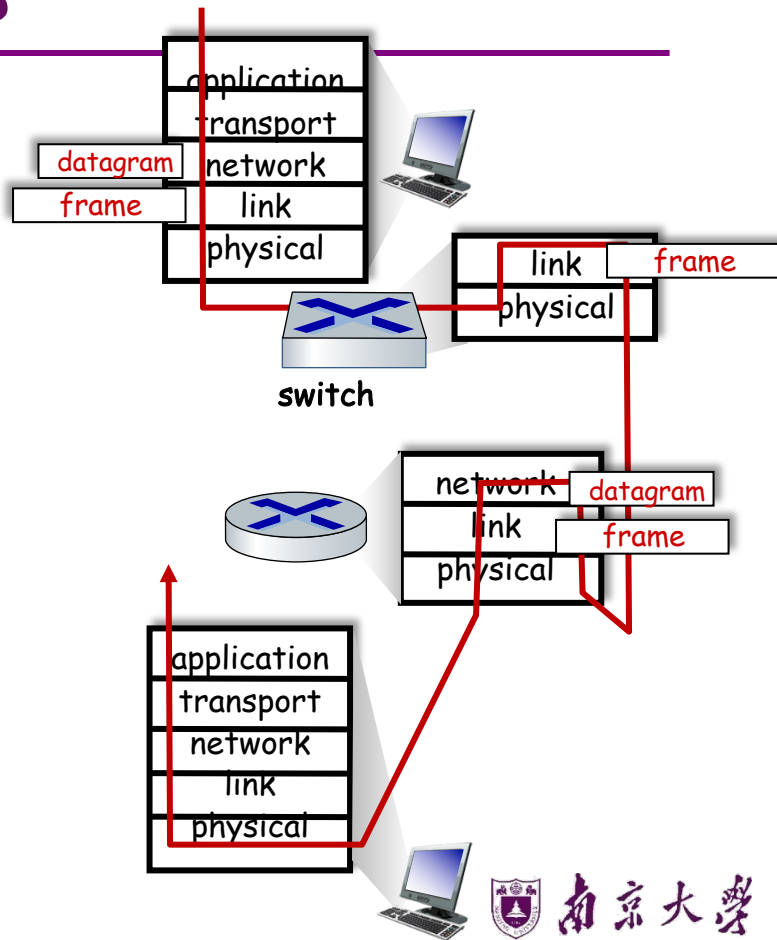
Switches vs. routers

both are store-and-forward:

- **routers:** network-layer devices (examine network-layer headers)
- **switches:** link-layer devices (examine link-layer headers)

both have forwarding tables:

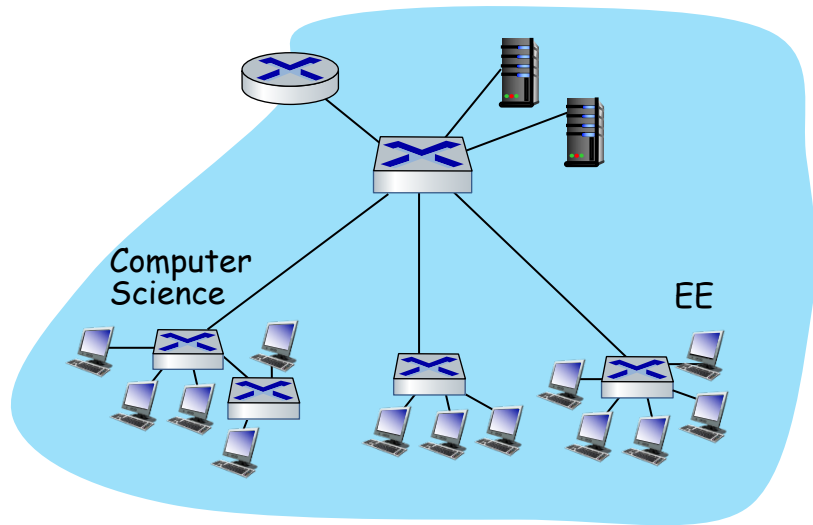
- **routers:** compute tables using routing algorithms, IP addresses
- **switches:** learn forwarding table using flooding, learning, MAC addresses





Virtual LANs (VLANs): motivation

Q: what happens as LAN sizes scale, users change point of attachment?



single broadcast domain:

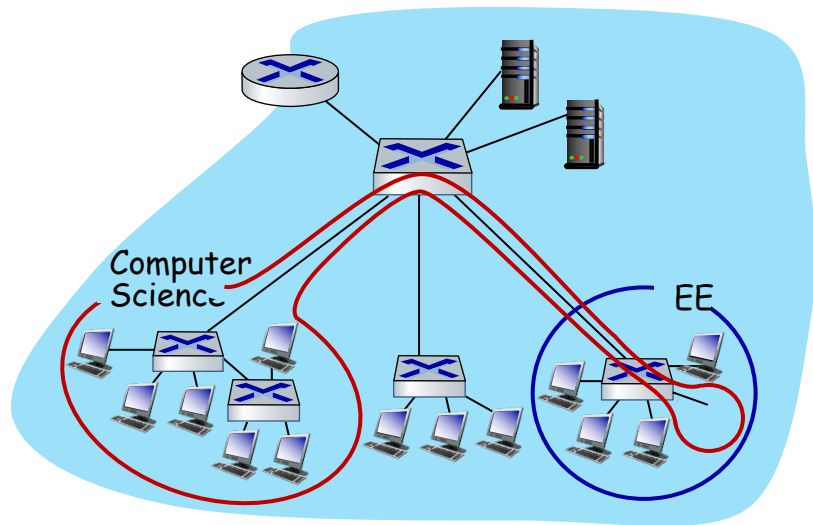
- **scaling:** all layer-2 broadcast traffic (ARP, DHCP, unknown MAC) must cross entire LAN
- efficiency, security, privacy issues





Virtual LANs (VLANs): motivation

Q: what happens as LAN sizes scale, users change point of attachment?



single broadcast domain:

- **scaling**: all layer-2 broadcast traffic (ARP, DHCP, unknown MAC) must cross entire LAN
- efficiency, security, privacy issues

administrative issues:

- CS user moves office to EE - **physically** attached to EE switch, but wants to remain **logically** attached to CS switch

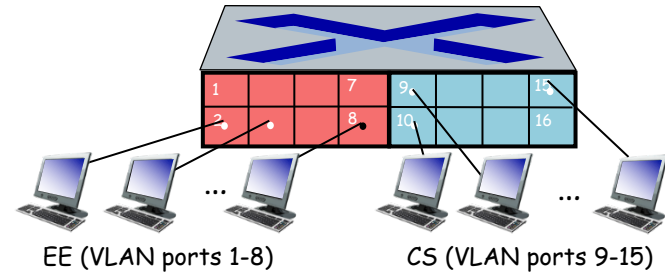


Port-based VLANs

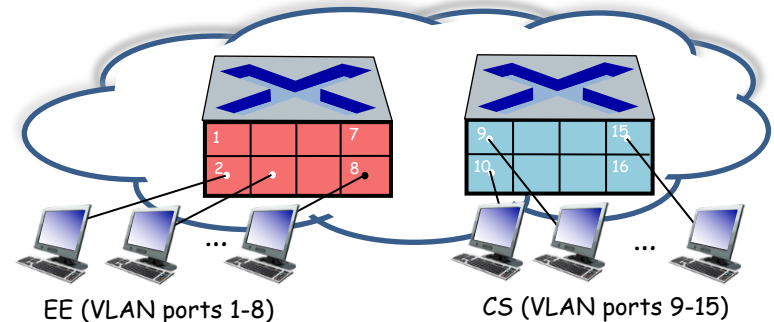
Virtual Local Area Network (VLAN)

switch(es) supporting VLAN capabilities can be configured to define multiple **virtual** LANS over single physical LAN infrastructure.

port-based VLAN: switch ports grouped (by switch management software) so that **single** physical switch



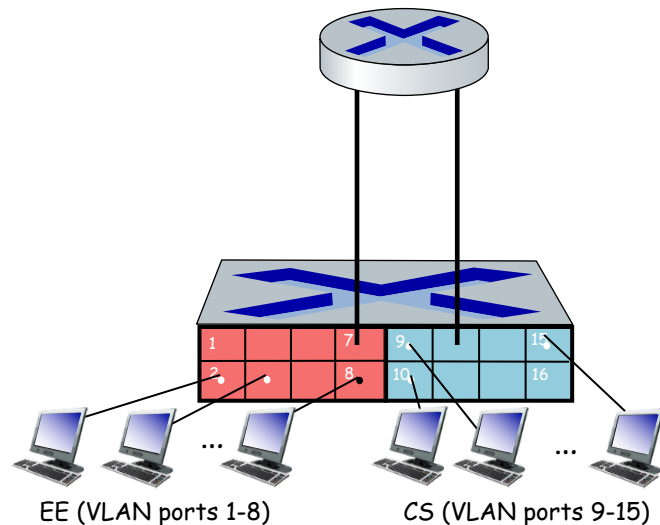
... operates as **multiple** virtual switches





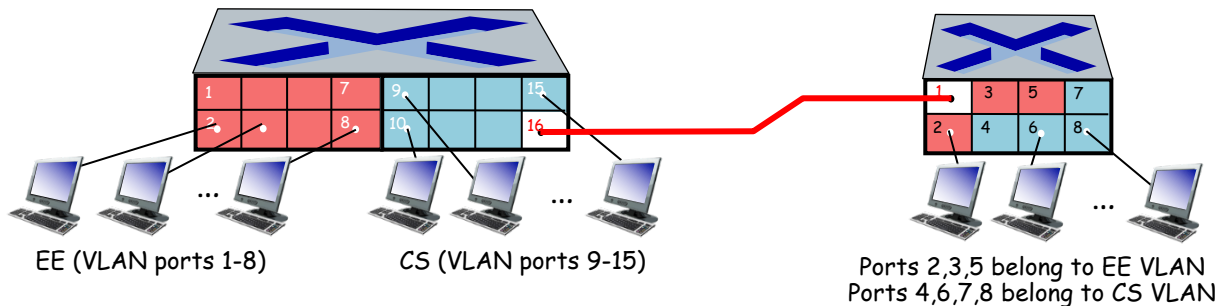
Port-based VLANs

- **traffic isolation:** frames to/from ports 1-8 can only reach ports 1-8
 - can also define VLAN based on MAC addresses of endpoints, rather than switch port
- **dynamic membership:** ports can be dynamically assigned among VLANs
- **forwarding between VLANs:** done via routing (just as with separate switches)
 - in practice vendors sell combined switches plus routers





VLANs spanning multiple switches

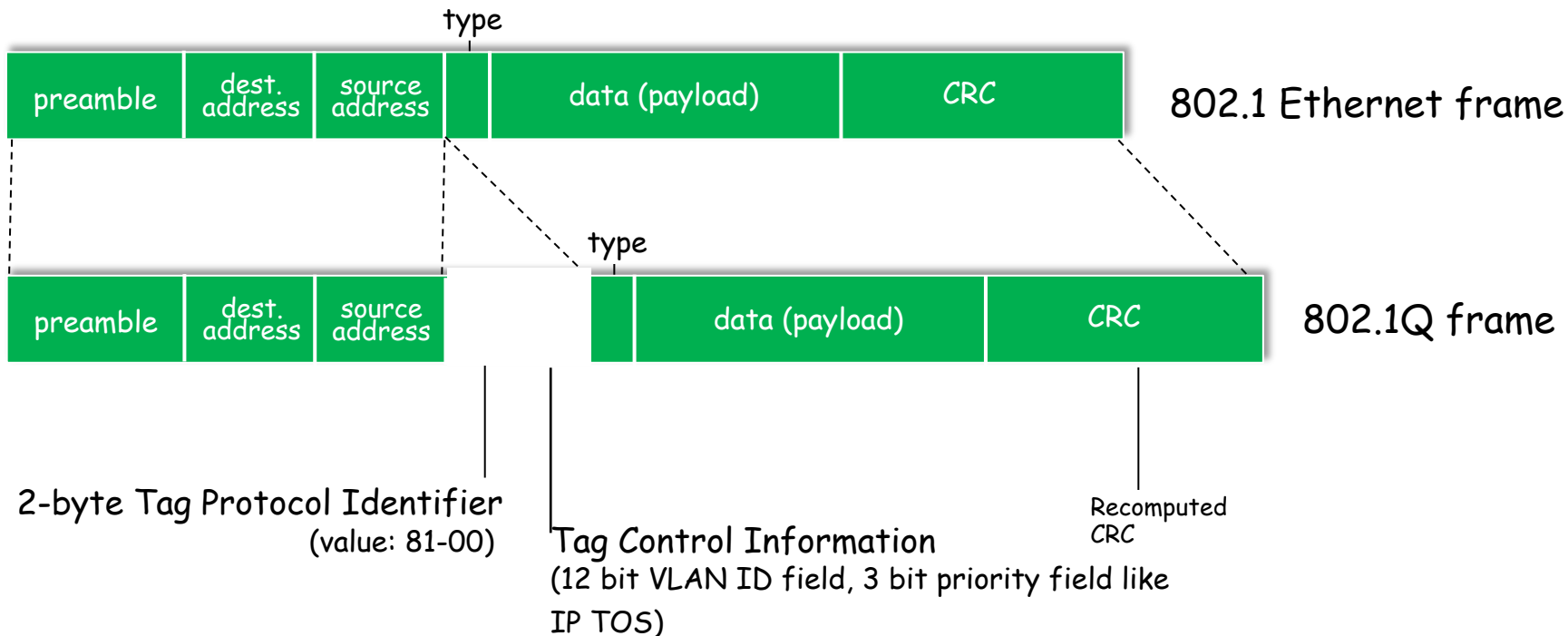


trunk port: carries frames between VLANs defined over multiple physical switches

- frames forwarded within VLAN between switches can't be vanilla 802.1 frames (must carry VLAN ID info)
- 802.1q protocol adds/removed additional header fields for frames forwarded between trunk ports

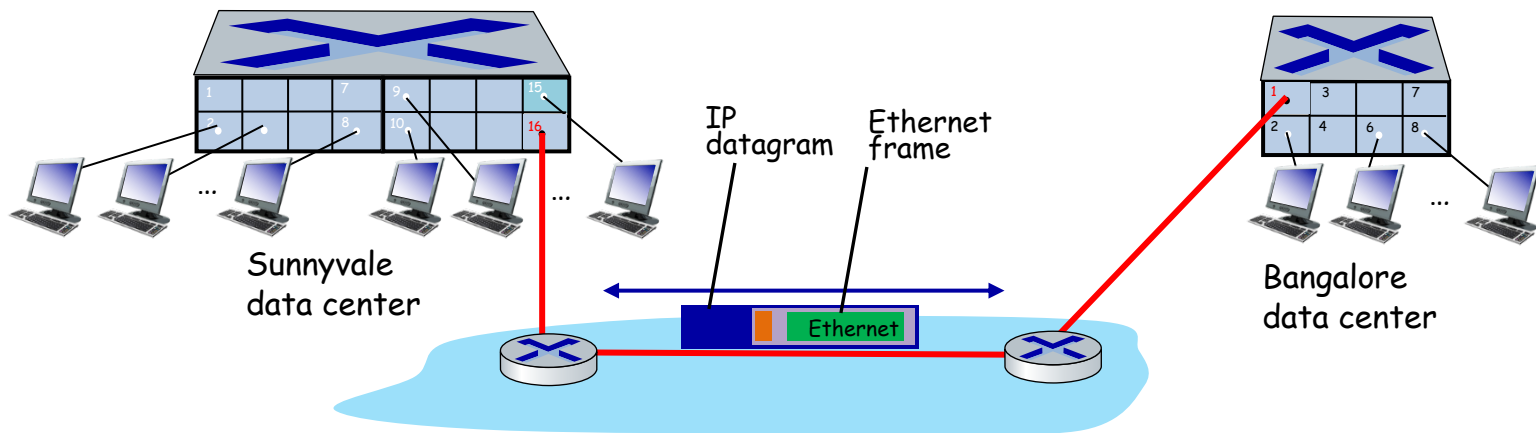


802.1Q VLAN frame format





EVPN: Ethernet VPNs (aka VXLANs)



Layer-2 Ethernet switches logically connected to each other (e.g., using IP as an **underlay**)

- Ethernet frames carried within IP datagrams between sites
- "tunneling scheme to **overlay Layer 2 networks on top of Layer 3 networks** ... runs over the existing networking infrastructure and provides a means to "stretch" a Layer 2 network." [RFC 7348]





Outline

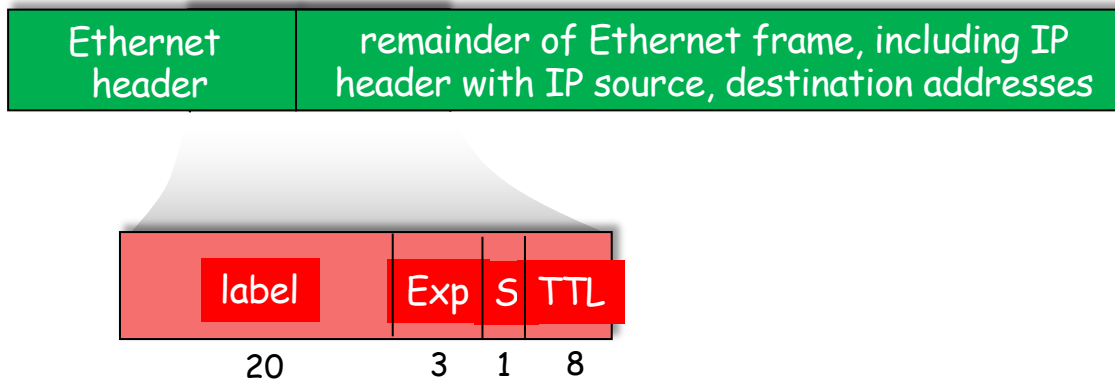
- Introduction
- Error detection, correction
- Multiple access protocols
- LANs
- Link virtualization: MPLS
- Data center networking
- A day in the life of a web request





Multiprotocol label switching (MPLS)

- **goal:** high-speed IP forwarding among network of MPLS-capable routers, using fixed length label (instead of shortest prefix matching)
 - faster lookup using fixed length identifier
 - borrowing ideas from Virtual Circuit (VC) approach
 - but IP datagram still keeps IP address!





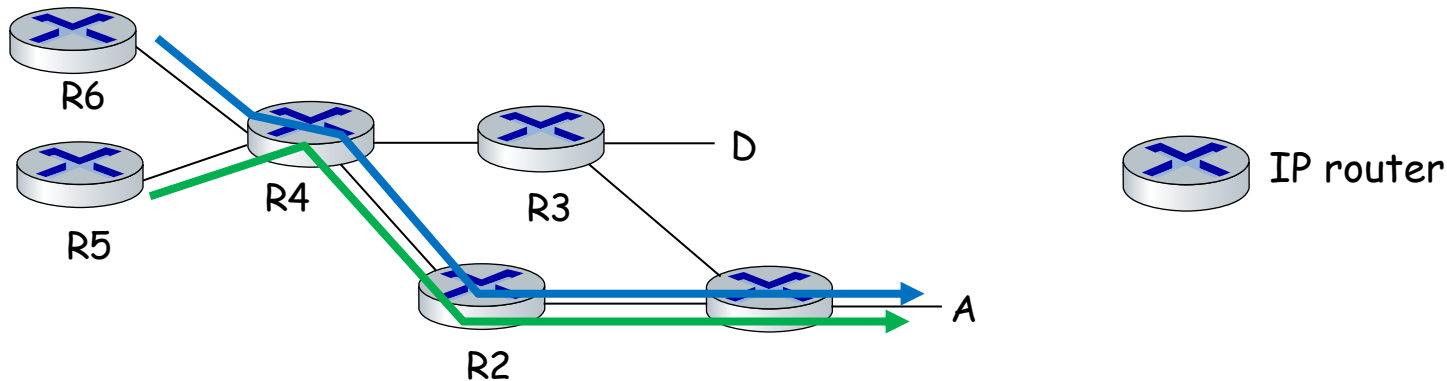
MPLS capable routers

- a.k.a. label-switched router
- forward packets to outgoing interface based only on label value (don't inspect IP address)
 - MPLS forwarding table distinct from IP forwarding tables
- **flexibility:** MPLS forwarding decisions can differ from those of IP
 - use destination and source addresses to route flows to same destination differently (traffic engineering)
 - re-route flows quickly if link fails: pre-computed backup paths





MPLS versus IP paths

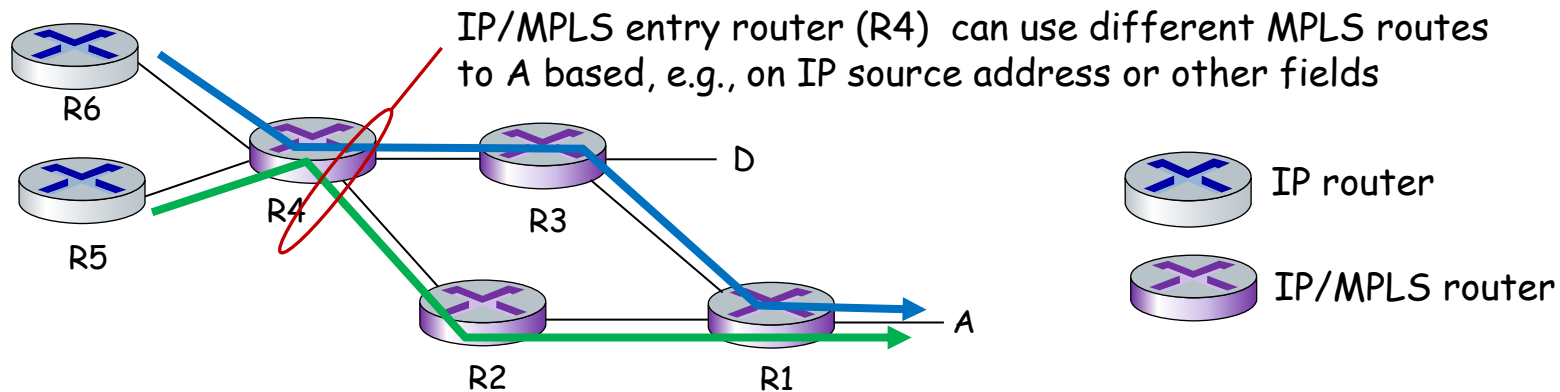


- **IP routing:** path to destination determined by destination address alone





MPLS versus IP paths



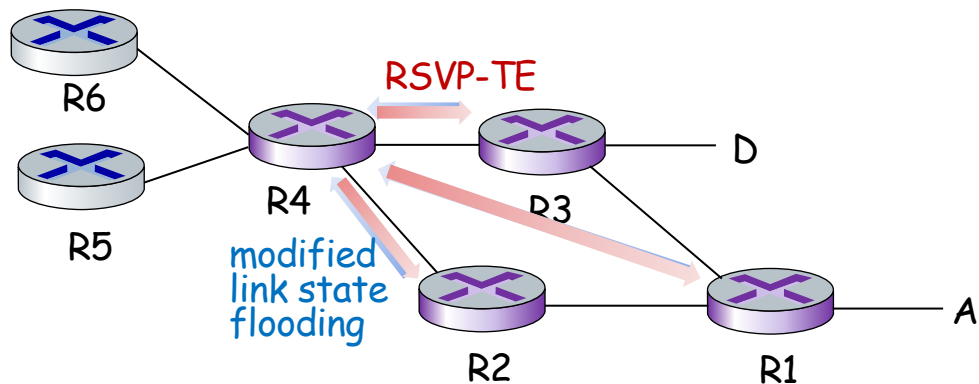
- **IP routing:** path to destination determined by destination address alone
- **MPLS routing:** path to destination can be based on source and destination address
 - flavor of generalized forwarding (MPLS 10 years earlier)
 - **fast reroute:** precompute backup routes in case of link failure





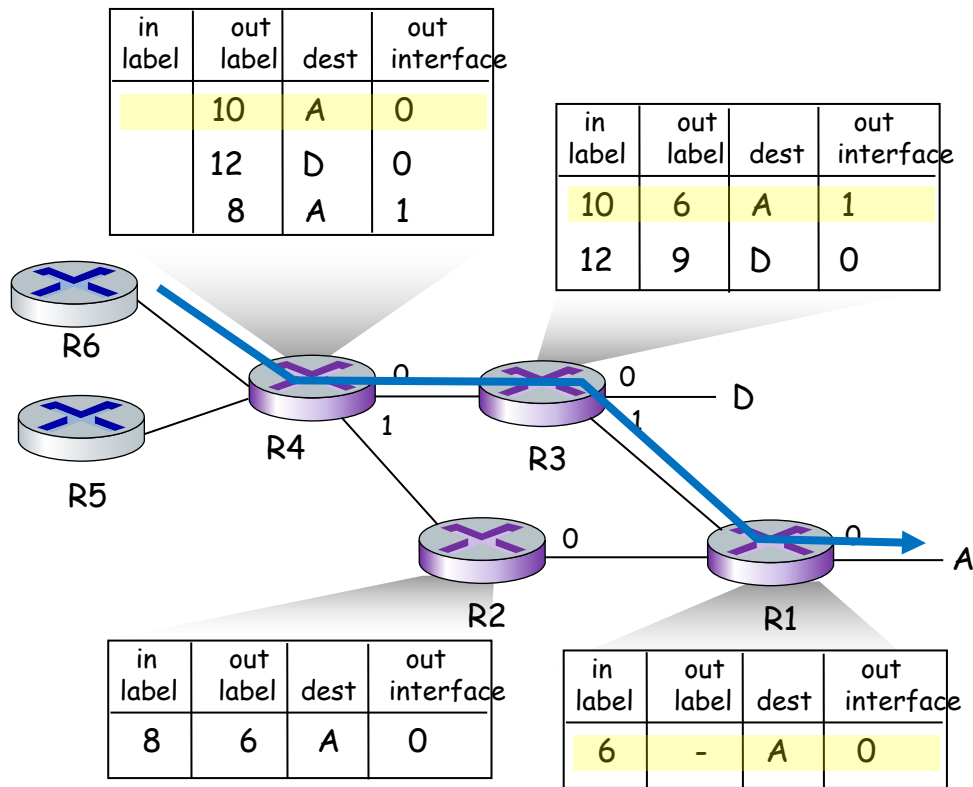
MPLS signaling

- modify OSPF, IS-IS link-state flooding protocols to carry info used by MPLS routing:
 - e.g., link bandwidth, amount of “reserved” link bandwidth
- entry MPLS router uses RSVP-TE signaling protocol to set up MPLS forwarding at downstream routers





MPLS forwarding tables





Outline

- Introduction
- Error detection, correction
- Multiple access protocols
- LANs
- Link virtualization: MPLS
- Data center networking
- A day in the life of a web request





Datacenter networks

10's to 100's of thousands of hosts, often closely coupled, in close proximity:

- e-business (e.g. Amazon)
- content-servers (e.g., Apple, Akamai, Microsoft)
- search engines, data mining (e.g., Google)

challenges:

- multiple applications, each serving massive numbers of clients
- reliability
- managing/balancing load, avoiding processing, networking, data bottlenecks

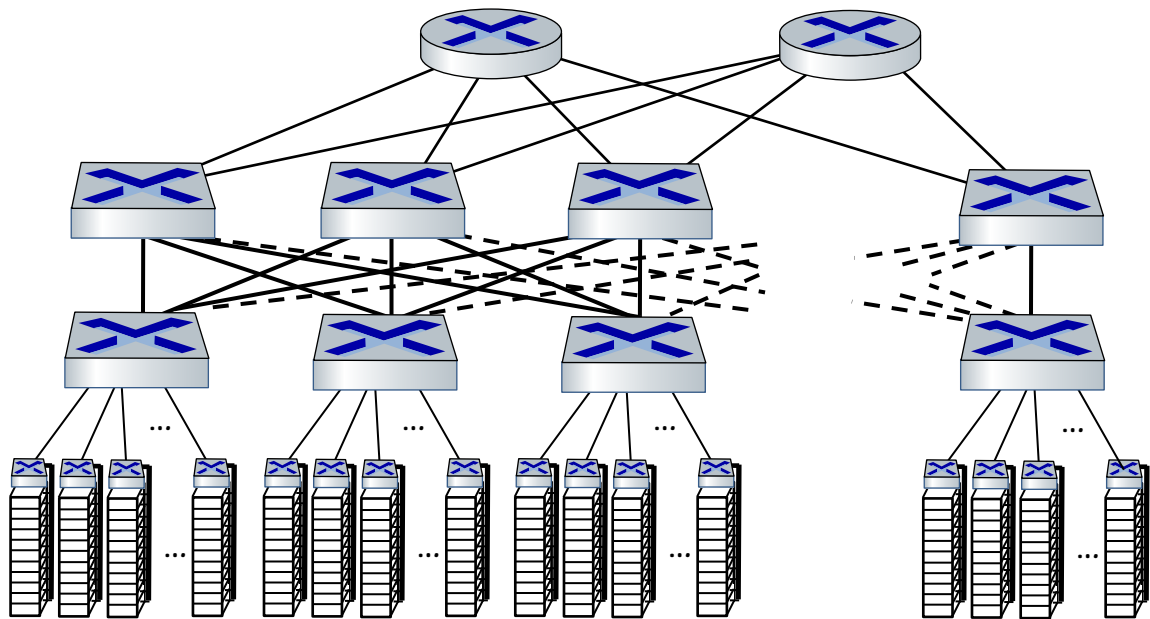


Inside a 40-ft Microsoft container, Chicago data center





Datacenter networks: network elements



Border routers

- connections outside datacenter

Tier-1 switches

- connecting to ~16 T-2s below

Tier-2 switches

- connecting to ~16 TORs below

Top of Rack (TOR) switch

- one per rack
- 100G-400G Ethernet to blades

Server racks

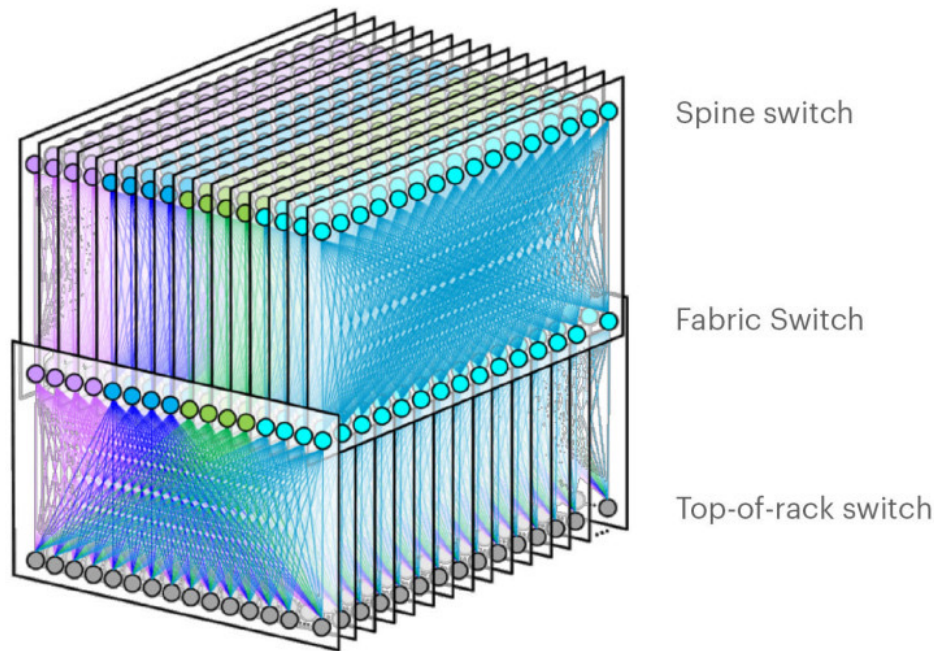
- 20- 40 server blades: hosts





Datacenter networks: network elements

Facebook F16 data center network topology:

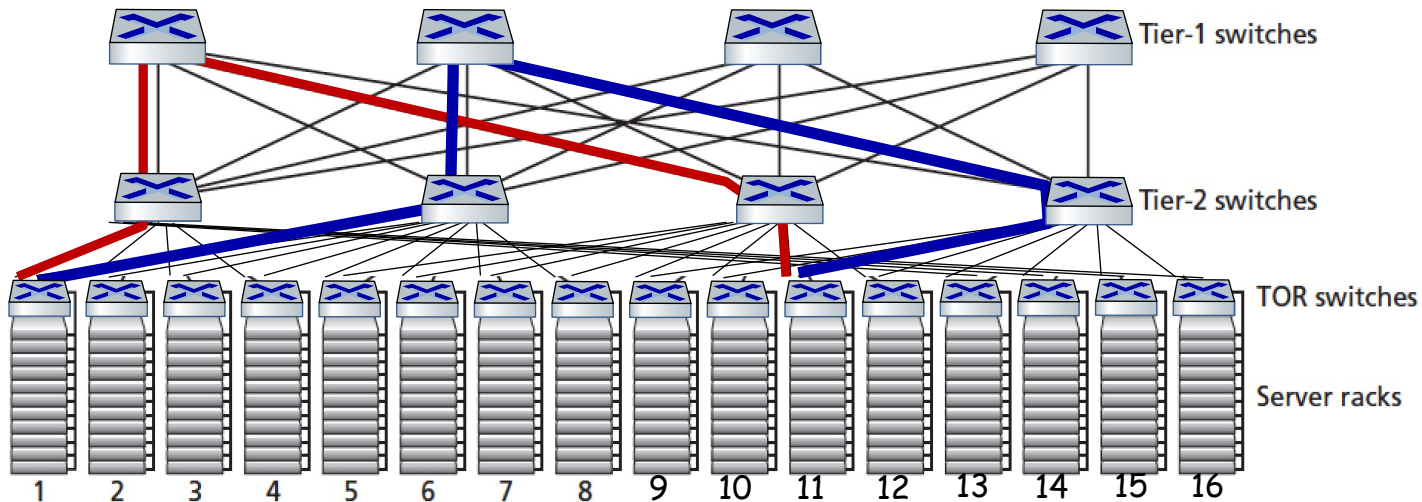


<https://engineering.fb.com/data-center-engineering/f16-minipack/> (posted 3/2019)



Datacenter networks: multipath

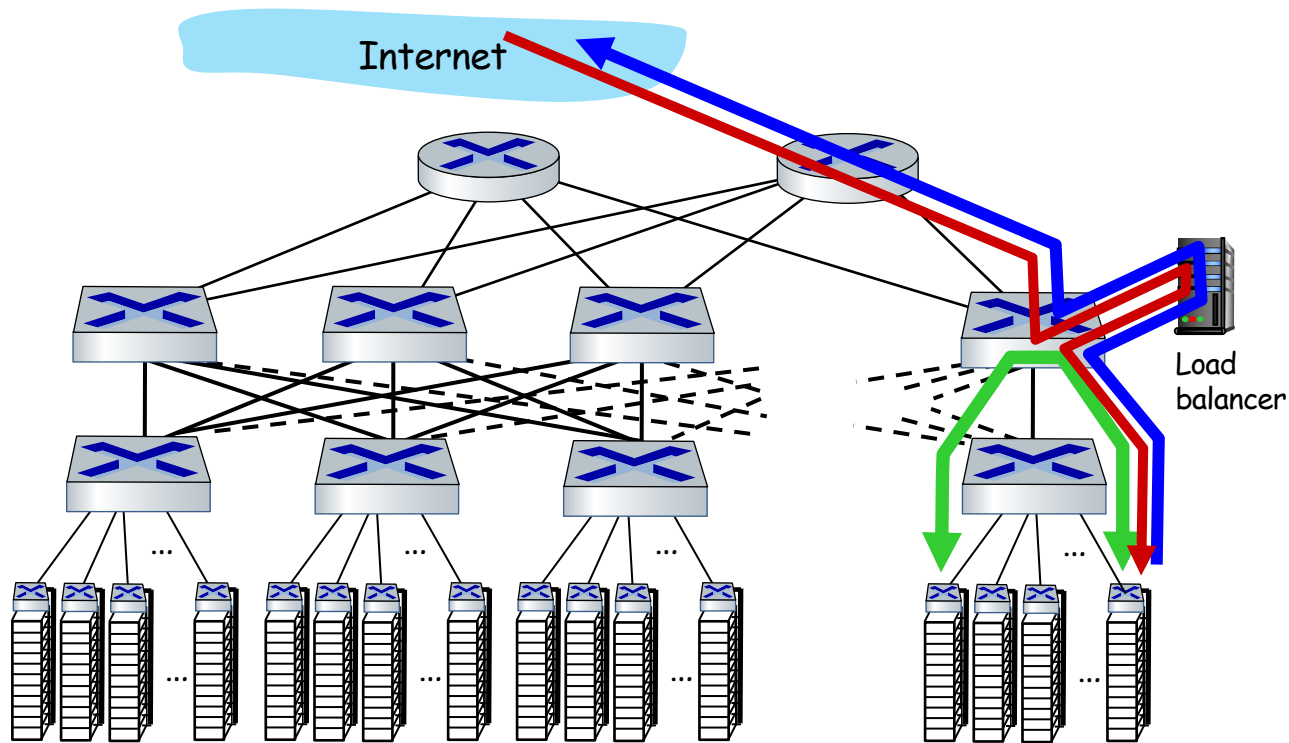
- rich interconnection among switches, racks:
 - increased throughput between racks (multiple routing paths possible)
 - increased reliability via redundancy



two **disjoint** paths highlighted between racks 1 and 11



Datacenter networks: application-layer routing



load balancer:
application-layer
routing

- receives external client requests
- directs workload within data center
- returns results to external client (hiding data center internals from client)





Datacenter networks: protocol innovations

- link layer:
 - RoCE: remote DMA (RDMA) over Converged Ethernet
- transport layer:
 - ECN (explicit congestion notification) used in transport-layer congestion control (DCTCP, DCQCN)
 - experimentation with hop-by-hop (backpressure) congestion control
- routing, management:
 - SDN widely used within/among organizations' datacenters
 - place related services, data as close as possible (e.g., in same rack or nearby rack) to minimize tier-2, tier-1 communication

Google Networking: Infrastructure and Selected Challenges (Slides:

<https://networkingchannel.eu/google-networking-infrastructure-and-selected-challenges/>



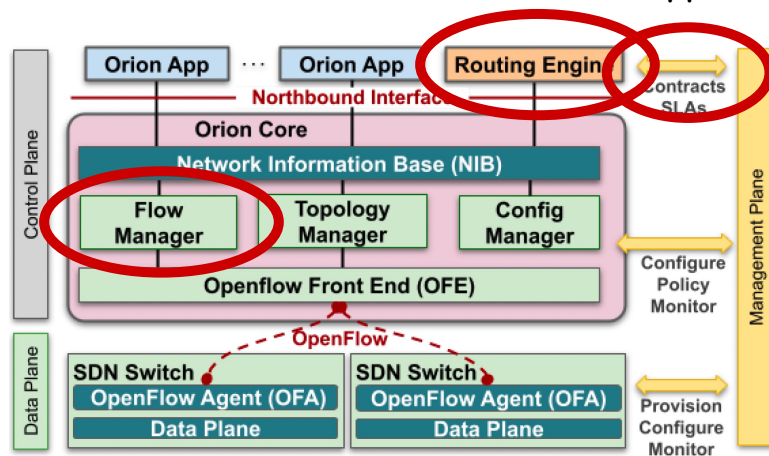
ORION: Google's new SDN control plane for internal datacenter (Jupiter) + wide area (B4) network

- **routing** (intradomain, iBGP), traffic engineering: implemented in applications on top of ORION core
- **edge-edge flow-based** controls (e.g., CoFlow scheduling) to meet contract SLAs
- **management**: pub-sub distributed microservices in Orion core, OpenFlow for switch signaling/monitoring

Note:

- no routing protocols, congestion control (partially) also managed by SDN rather than by protocol
- are protocols dying?

Orion SDN architecture and core apps





Outline

- Introduction
- Error detection, correction
- Multiple access protocols
- LANs
- Link virtualization: MPLS
- Data center networking
- A day in the life of a web request

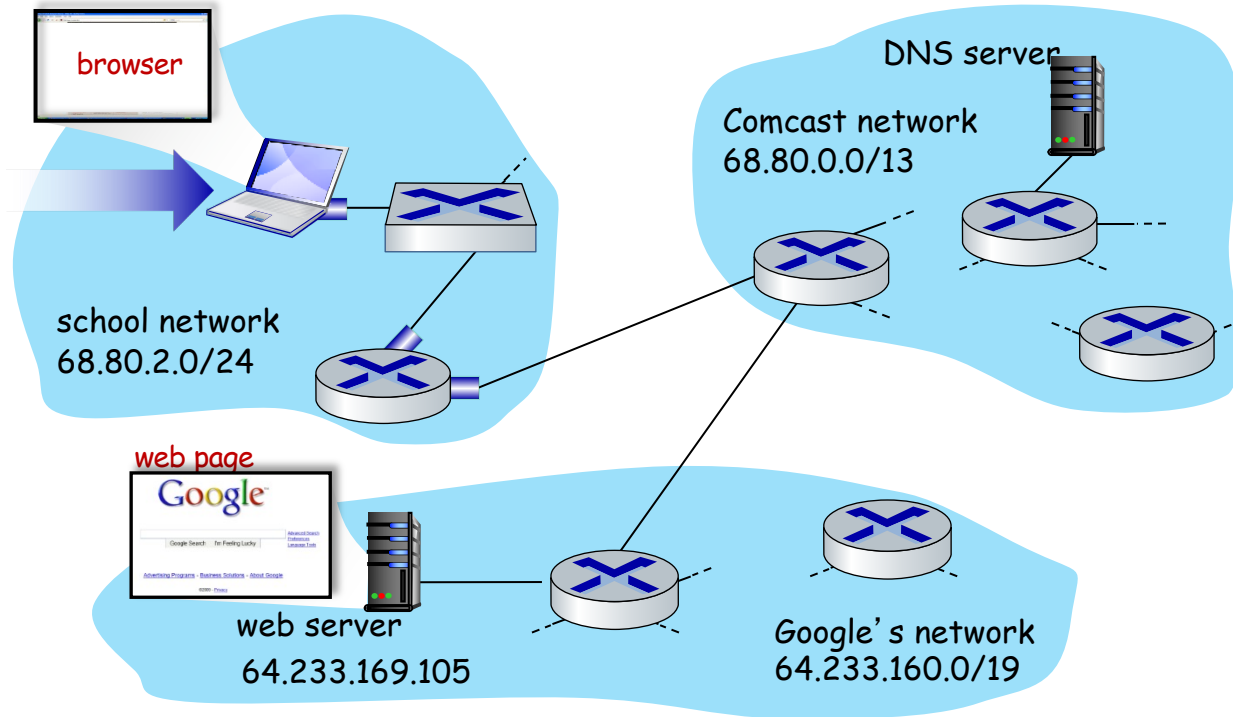


Synthesis: a day in the life of a web request

- our journey down the protocol stack is now complete!
 - application, transport, network, link
- putting-it-all-together: synthesis!
 - **goal:** identify, review, understand protocols (at all layers) involved in seemingly simple scenario: requesting www page
 - **scenario:** student attaches laptop to campus network, requests/receives www.google.com



A day in the life: scenario



scenario:

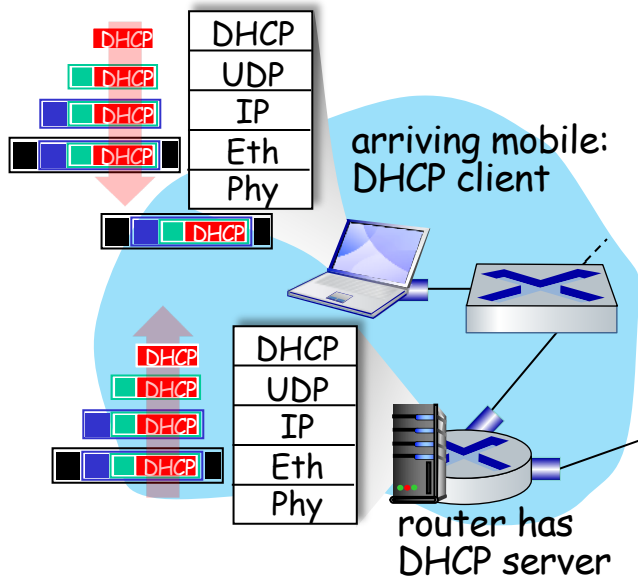
- arriving mobile client attaches to network ...
- requests web page:
`www.google.com`

Sounds
simple! 





A day in the life: connecting to the Internet

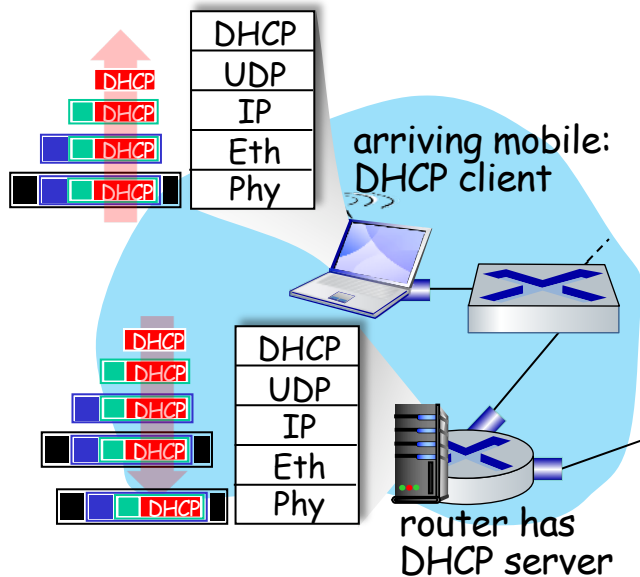


- connecting laptop needs to get its own IP address, addr of first-hop router, addr of DNS server: use **DHCP**
- DHCP request **encapsulated** in **UDP**, encapsulated in **IP**, encapsulated in **802.3 Ethernet**
- Ethernet frame **broadcast** (dest: FFFFFFFFFFFFFFFF) on LAN, received at router running **DHCP** server
- Ethernet **de-muxed** to IP de-muxed, UDP de-muxed to DHCP





A day in the life: connecting to the Internet



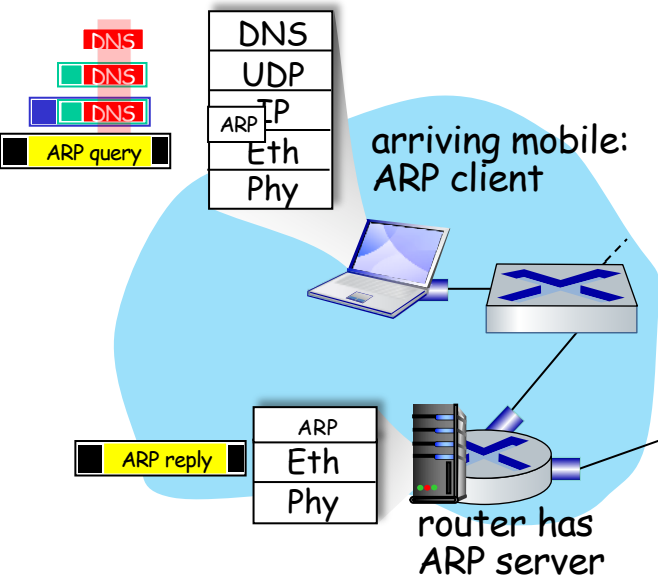
- DHCP server formulates **DHCP ACK** containing client's IP address, IP address of first-hop router for client, name & IP address of DNS server
- encapsulation at DHCP server, frame forwarded (**switch learning**) through LAN, demultiplexing at client
- DHCP client receives DHCP ACK reply

Client now has IP address, knows name & addr of DNS server,
IP address of its first-hop router





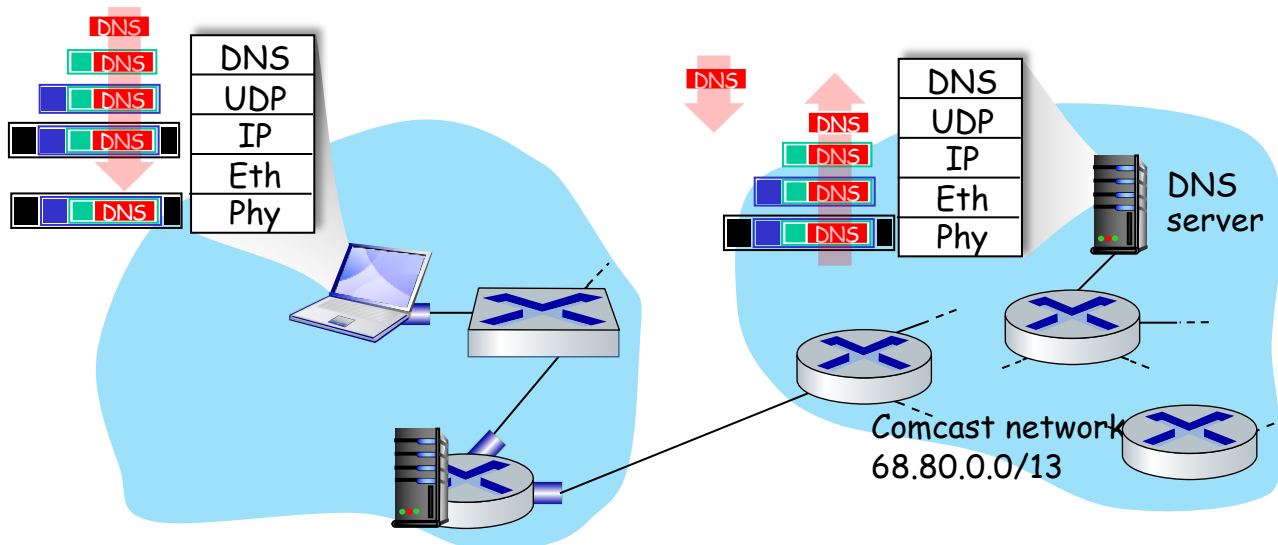
A day in the life... ARP (before DNS, before HTTP)



- before sending **HTTP** request, need IP address of **www.google.com**: **DNS**
- DNS query created, encapsulated in UDP, encapsulated in IP, encapsulated in Eth. To send frame to router, need MAC address of router interface: **ARP**
- **ARP query** broadcast, received by router, which replies with **ARP reply** giving MAC address of router interface
- client now knows MAC address of first hop router, so can now send frame containing DNS query



A day in the life... using DNS



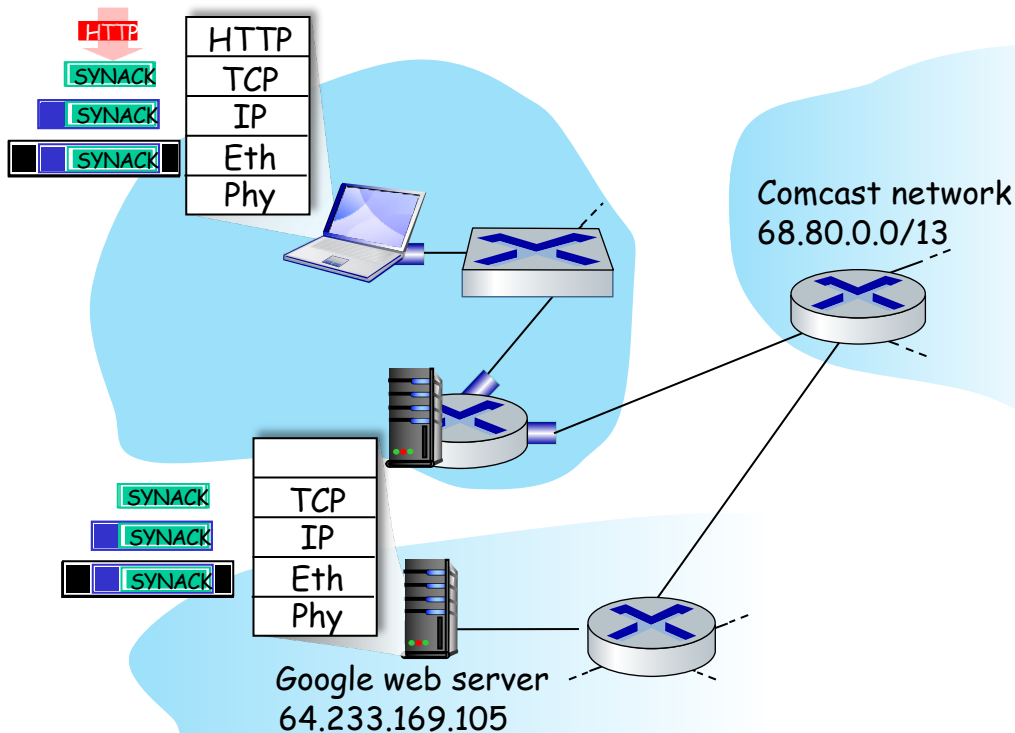
- de-muxed to DNS
- DNS replies to client with IP address of www.google.com

- IP datagram containing DNS query forwarded via LAN switch from client to 1st hop router
- IP datagram forwarded from campus network into Comcast network, routed (tables created by **RIP**, **OSPF**, **IS-IS** and/or **BGP** routing protocols) to DNS server





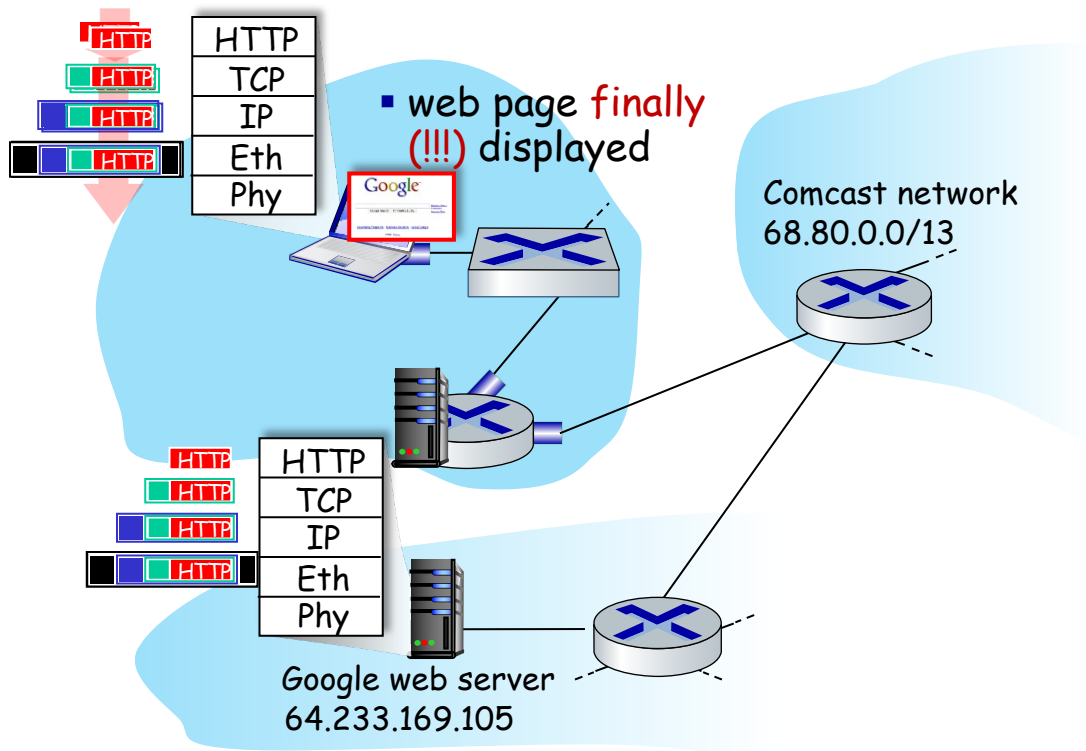
A day in the life...TCP connection carrying HTTP



- to send HTTP request, client first opens **TCP socket** to web server
- TCP **SYN segment** (step 1 in TCP 3-way handshake) inter-domain routed to web server
- web server responds with **TCP SYNACK** (step 2 in TCP 3-way handshake)
- TCP **connection established!**



A day in the life... HTTP request/reply



- HTTP request sent into TCP socket
- IP datagram containing HTTP request routed to www.google.com
- web server responds with HTTP reply (containing web page)
- IP datagram containing HTTP reply routed back to client





Summary

- principles behind data link layer services:
 - error detection, correction
 - sharing a broadcast channel: multiple access
 - link layer addressing
- instantiation, implementation of various link layer technologies
 - Ethernet
 - switched LANS, VLANs
 - virtualized networks as a link layer: MPLS
- synthesis: a day in the life of a web request





提问

Q & A

孟晴开

智能软件与工程学院

苏州校区南雍楼西区228

qkmeng@nju.edu.cn , <https://mengqingkai.github.io/>



南京大學
NANJING UNIVERSITY